

Introduction

The Xilinx® LogiCORE™ IP Controller Area Network (CAN) product specification defines the architecture and features of the Xilinx CAN controller core. This document also defines the addressing and functionality of the various registers in the design, in addition to describing the user interface. The scope of this document does not extend to describing the CAN protocol and assumes knowledge of the specifications described in the Reference Documents section.

Features

- **Conforms to the ISO 11898 -1, CAN 2.0A, and CAN 2.0B standards**
- Supports Industrial (I) and Extended Temperature Range (Q) grade device support
- Supports both standard (11-bit identifier) and extended (29-bit identifier) frames
- Supports bit rates up to 1 Mb/s
- Transmit message FIFO with a user-configurable depth of up to 64 messages
- Transmit prioritization through one High-Priority Transmit buffer
- Automatic re-transmission on errors or arbitration loss
- Receive message FIFO with a user- configurable depth of up to 64 messages
- Acceptance filtering (through a user-configurable number) of up to four acceptance filters
- Sleep Mode with automatic walk-up
- Loop Back Mode for diagnostic applications
- Maskable Error and Status Interrupts
- Readable Error Counters

LogiCORE IP Facts Table	
Core Specifics	
Supported Device Family ⁽¹⁾	Virtex-7 Kintex™-7 Artix™-7 Zynq™-7000 Virtex-6 Spartan-6/XA
Supported User Interfaces	AXI4-Lite
Resources	
See Table 40 and Table 41 .	
Provided with Core	
Documentation	Product Specification
Design Files	VHDL
Example Design	Not Provided
Test Bench	Not Provided
Constraints File	None
Simulation Model	None
Tested Design Tools	
Design Entry Tools	XPS 13.2
Simulation	Mentor Graphics ModelSim ⁽²⁾
Synthesis Tools	XST 13.2
Support	
Provided by Xilinx, Inc.	

1. For a complete listing of supported devices, see [IDS Embedded Edition Derivative Device Support](#) for this core.
2. For the supported versions of the tools, see the [ISE Design Suite 13: Release Notes Guide](#).

Functional Description

Figure 1 illustrates the high-level architecture of the CAN core. The CAN core requires an external 3.3 V compatible PHY device. Descriptions of the submodules are given in the following sections.

Configuration Registers

Table 6 defines the configuration registers. This module allows for read and write access to the registers through the external micro-controller interface.

Transmit and Receive Messages

Separate storage buffers exist for transmit (TX FIFO) and receive (RX FIFO) messages through a FIFO structure. The depth of each buffer is individually configurable up to a maximum of 64 messages.

Transmit High Priority Buffer

The Transfer High Priority Buffer (TX HPB) provides storage for one transmit message. Messages written on this buffer have maximum transmit priority. They are queued for transmission immediately after the current transmission is complete, preempting any message in the TX FIFO.

Acceptance Filters

Acceptance Filters sort incoming messages with the user-defined acceptance mask and ID registers to determine whether to store messages in the RX FIFO, or to acknowledge and discard them. The number of acceptance filters can be configured from 0 to 4. Messages passed through acceptance filters are stored in the RX FIFO.

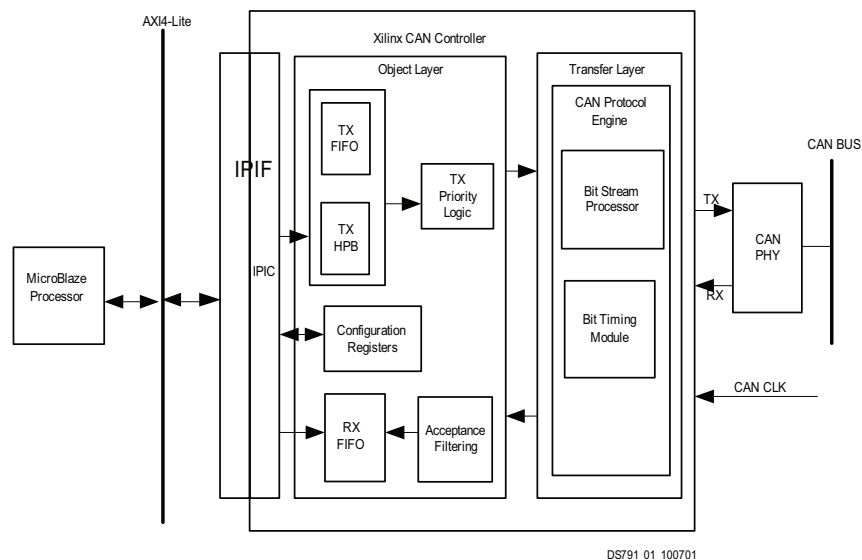


Figure 1: AXI CAN Block Diagram

Protocol Engine

The CAN protocol engine consists primarily of the Bit Timing Logic (BTL) and the Bit Stream Processor (BSP) modules. Figure 2 illustrates a block diagram of the CAN protocol engine.

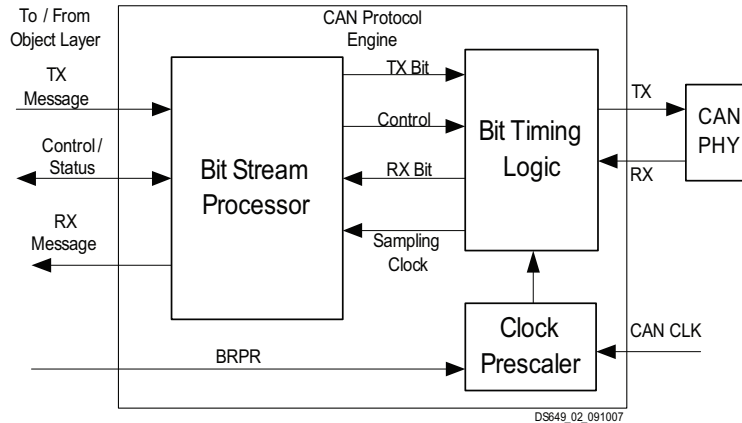


Figure 2: CAN Protocol Engine

Bit Timing Logic

The primary functions of the Bit Timing Logic (BTL) module include:

- Synchronizing the CAN controller to CAN traffic on the bus
- Sampling the bus and extracting the data stream from the bus during reception
- Inserting the transmit bitstream onto the bus during transmission
- Generating a sampling clock for the BSP module state machine

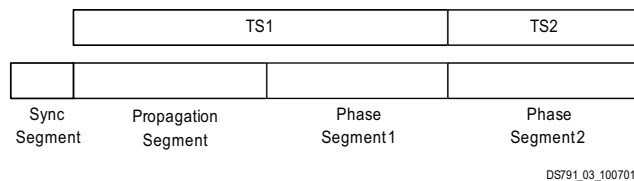


Figure 3: CAN Bit Timing

As illustrated in Figure 3, the CAN bit time is divided into four parts:

- Sync segment
- Propagation segment
- Phase segment 1
- Phase segment 2

These bit time parts are comprised of a number of smaller segments of equal length called *time quanta* (tq). The length of each time quantum is equal to the quantum clock time period (period = tq). The quantum clock is generated internally by dividing the incoming oscillator clock by the baud rate pre-scaler. The pre-scaler value is passed to the BTL module through the Baud Rate Prescaler (BRPR) register.

The propagation segment and phase segment 1 are joined and called 'time segment1' (TS1), while phase segment 2 is called 'time segment2' (TS2). The number of time quanta in TS1 and TS2 vary with different networks and are specified in the Bit Timing Register (BTR), which is passed to the BTL module. The Sync segment is always one time quantum long.

The BTL state machine runs on the quantum clock. During the SOF bit of every CAN frame, the state machine is instructed by the Bit Stream Processor module to perform a *hard* sync, forcing the recessive (r) to dominant edge (d) to lie in the sync segment. During the rest of the recessive-to-dominant edges in the CAN frame, the BTL is prompted to perform resynchronization.

During resynchronization, the BTL waits for a recessive-to-dominant edge and after that occurs, it calculates the time difference (number of tq's) between the edge and the nearest sync segment. To compensate for this time difference and to force the sampling point to occur at the correct instant in the CAN bit time, the BTL modifies the length of phase segment 1 or phase segment 2.

The maximum amount by which the phase segments can be modified is dictated by the Synchronization Jump Width (SJW) parameter, which is also passed to the BTL through the BTR. The length of the bit time of subsequent CAN bits are unaffected by this process. This synchronization process corrects for propagation delays and oscillator mismatches between the transmitting and receiving nodes.

After the controller is synchronized to the bus, the state machine waits for a time period of TS1 and then samples the bus, generating a digital '0' or '1'. This is passed on to the BSP module for higher level tasks.

Bit Stream Processor

The Bit Stream Processor (BSP) module performs several MAC/LLC functions during reception (RX) and transmission (TX) of CAN messages. The BSP receives a message for transmission from either the TX FIFO or the TX HPB and performs the following functions before passing the bitstream to BTL.

- Serializing the message
- Inserting stuff bits, CRC bits, and other protocol defined fields during transmission

During transmission, the BSP simultaneously monitors RX data and performs bus arbitration tasks. It then transmits the complete frame when arbitration is won, and retrying when arbitration is lost.

During reception, the BSP removes Stuff bits, CRC bits, and other protocol fields from the received bitstream. The BSP state machine also analyses bus traffic during transmission and reception for Form, CRC, ACK, Stuff, and Bit violations. The state machine then performs error signaling and error confinement tasks. The CAN controller does not voluntarily generate overload frames but does respond to overload flags detected on the bus.

This module determines the error state of the CAN controller: Error Active, Error Passive, or Bus-off. When TX or RX errors are observed on the bus, the BSP updates the transmit and receive error counters according to the rules defined in the CAN 2.0 A, CAN 2.0 B and ISO 11898-1 standards. Based on the values of these counters, the error state of the CAN controller is updated by the BSP.

I/O Signals

The AXI CAN I/O signals are listed and described in [Table 1](#)

Table 1: I/O Signal Description

Port	Signal Name	Interface	Signal Type	Initial State	Description
AXI Global System Signals					
P1	S_AXI_ACLK	AXI	I	N/A	AXI Clock
P2	S_AXI_ARESET_N	AXI	I	N/A	AXI Reset (active Low)
AXI Write Address Channel Signals					
P3	S_AXI_AWADDR[C_S_AXI_ADDR_WIDTH- 1:0]	AXI	I	N/A	AXI Write address. The write address bus gives the address of the write transaction.
P4	S_AXI_AWVALID	AXI	I	N/A	Write address valid. This signal indicates that valid write address and control information are available.
P5	S_AXI_AWREADY	AXI	O	N/A	Write address ready. This signal indicates that the slave is ready to accept an address and associated control signals.
AXI Write Data Channel Signals					
P6	S_AXI_WDATA[C_S_AXI_DATA_WIDTH - 1: 0]	AXI	I	N/A	Write Data
P7	S_AXI_WSTB[C_S_AXI_DATA_WIDTH/8- 1:0]	AXI	I	N/A	Write strobes. This signal indicates which byte lanes to update in memory.
P8	S_AXI_WVALID	AXI	I	N/A	Write valid. This signal indicates that valid write data and strobes are available.
P9	S_AXI_WREADY	AXI	O	0x0	Write ready. This signal indicates that the slave can accept the write data.
AXI Write Response Channel Signal					
P10	S_AXI_BRESP[1:0]	AXI	O	0x0	Write response. This signal indicates the status of the write transaction. "00"- OKAY "10"- SLVERR "11"- DECERR
P11	S_AXI_BVALID	AXI	O	0x0	Write response valid. This signal indicates that a valid write response is available
P12	S_AXI_BREADY	AXI	I	0x1	Response ready. This signal indicates that the master can accept the response information

Table 1: I/O Signal Description (Cont'd)

AXI Read Address Channel Signal					
P13	S_AXI_ARADDR[C_S_AXI_ADDR_WIDTH - 1:0]	AXI	I	N/A	Read address. The read address bus gives the address of a read transaction.
P14	S_AXI_ARVALID	AXI	I	N/A	Read address valid. This signal indicates, when HIGH, that the read address and control information is valid and remains stable until the address acknowledgement signal, S_AXI_ARREADY, is high.
P15	S_AXI_ARREADY	AXI	O	0x1	Read address ready. This signal indicates that the slave is ready to accept an address and associated control signals.
AXI Read Data Channel Signal					
P16	S_AXI_RDATA[C_S_AXI_DATA_WIDTH -1:0]	AXI	O	0x0	Read data
P17	S_AXI_RRESP[1:0]	AXI	O	0x0	Read response. This signal indicates the status of the read transfer. "00"- OKAY "10"- SLVERR "11"- DECERR
P18	S_AXI_RVALID	AXI	O	0x0	Read valid. This signal indicates that the required read data is available and the read transfer can complete.
P19	S_AXI_RREADY	AXI	I	0x1	Read ready. This signal indicates that the master can accept the read data and response information.
CAN Signals					
P20	CAN_CLK	CAN	I		Oscillator Clock input (max value of 24 MHz)
P21	CAN_PHY_RX	CAN	I		CAN bus receive signal from PHY
P22	CAN_PHY_TX	CAN	O	1	CAN bus transmit signal to PHY
P23	IP2Bus_IntrEvent	CAN	O	1	Interrupt line from CAN

Note: S_AXI_Clk frequency must be greater than or equal to CAN_CLK frequency.

Register Bit Ordering

All registers use big-endian bit ordering where bit-0 is MSB and bit-31 is LSB. Table 2 shows the bit ordering.

Table 2: Register Bit Ordering

0	1	2		29	30	31
MSB						LSB

Controller Design Parameters

To obtain a CAN controller uniquely tailored to meet the minimum system requirements, certain features are parameterized. This results in a design using only the required resources, providing the best possible performance. [Table 3](#) shows the CAN controller features that can be parameterized.

In addition to the parameters listed in this table, there are also parameters that are inferred for each AXI interface in the EDK tools. Through the design, these EDK-inferred parameters control the behavior of the AXI Interconnect. For a complete list of the interconnect settings related to the AXI interface, see *AXI Interconnect IP Data Sheet* (DS768).

Table 3: Design Parameters

Generic	Feature/Description	Parameter Name	Allowable Values	Default Values	VHDL Type
System Parameters					
G1	Target FPGA Family	C_FAMILY	Spartan-6 & Virtex-6		string
G2	Base Address of the Xilinx CAN Controller	C_S_AXI_BASEADDR	Valid address	See footnotes (1) and (2)	std_logic_vector
G3	High Address of the Xilinx CAN Controller	C_S_AXI_HIGHADDR	Valid address	See footnotes (1) and (2)	std_logic_vector
CAN Parameters					
G4	Number of Acceptance Filters used	C_CAN_NUM_ACF	0 - 4	0	integer
G5	Depth of the RX FIFO	C_CAN_RX_DPTH	2,4,8,16,32,64	2	integer
G6	Depth of the TX FIFO	C_CAN_TX_DPTH	2,4,8,16,32,64	2	integer
AXI Parameters					
G7	AXI Address bus width	C_S_AXI_ADDR_WIDTH	32	32	integer
G8	AXI Data bus width	C_S_AXI_DATA_WIDTH	32	32	integer

1. Address range specified by C_S_AXI_BASEADDR and C_S_AXI_HIGHADDR must be at least 0x100 and must be power of 2. C_S_AXI_BASEADDR must be multiple of the range, where the range is C_S_AXI_HIGHADDR - C_S_AXI_BASEADDR + 1. Also make sure that LSB 8 bits of the C_S_AXI_BASEADDR to be zero.
2. No default value is specified to ensure that the actual value is set, that is, if the value is not set, a compiler error is generated. The address range must be at least 0x00FF. For example, C_S_AXI_BASEADDR = 0x80000000, C_S_AXI_HIGHADDR = 0x800000FF.

The width of some of the AXI CAN device signals depends on parameters selected in the design. The dependencies between the AXI CAN device design parameters and I/O signals are shown in [Table 4](#).

Table 4: Parameter Port Dependencies

Generic or Port	Name	Affects	Depends	Relationship Description
Design Parameters				
G3	C_S_AXI_HIGHADDR		G2	Address range pair dependency
G7	C_S_AXI_ADDR_WIDTH	P3, P13		Defines the width of the ports
G8	C_S_AXI_DATA_WIDTH	P6, P7, P16		Defines the width of the ports
I/O Signals				
P3	S_AXI_AWADDR[C_S_AXI_ADDR_WIDTH- 1:0]	-	G7	Port width depends on the generic C_S_AXI_ADDR_WIDTH.
P6	S_AXI_WDATA[C_S_AXI_DATA_WIDTH - 1: 0]	-	G8	Port width depends on the generic C_S_AXI_DATA_WIDTH.
P7	S_AXI_WSTB[C_S_AXI_DATA_WIDTH/8- 1:0]	-	G8	Port width depends on the generic C_S_AXI_DATA_WIDTH.
P13	S_AXI_ARADDR[C_S_AXI_ADDR_WIDTH - 1:0]	-	G7	Port width depends on the generic C_S_AXI_ADDR_WIDTH.
P16	S_AXI_RDATA[C_S_AXI_DATA_WIDTH - 1:0]	-	G8	Port width depends on the generic C_S_AXI_DATA_WIDTH.

Operational Modes

The CAN controller supports these modes of operation:

- Configuration
- Normal
- Sleep
- Loop Back

[Table 5](#) contains the CAN Controller modes of operation and the corresponding control and status bits. Inputs that affect the mode transitions are discussed in [Configuration Register Descriptions, page 13](#).

Table 5: CAN Controller Modes of Operation

S_AXI_ARESET_N	SRST Bit (SRR)	CEN Bit (SRR)	LBACK Bit (MSR)	SLEEP Bit (MSR)	Status Register Bits (SR) (Read Only)				Operation Mode
					CONFIG	LBACK	SLEEP	NORMAL	
'0'	X	X	X	X	'1'	'0'	'0'	'0'	Core is Reset
'1'	'1'	X	X	X	'1'	'0'	'0'	'0'	Core is Reset
'1'	'0'	'0'	X	X	'1'	'0'	'0'	'0'	Configuration Mode
'1'	'0'	'1'	'1'	X	'0'	'1'	'0'	'0'	Loop Back Mode
'1'	'0'	'1'	'0'	'1'	'0'	'0'	'1'	'0'	Sleep Mode
'1'	'0'	'1'	'0'	'0'	'0'	'0'	'0'	'1'	Normal Mode

Configuration Mode

The CAN controller enters the Configuration mode when any of the following actions are performed, regardless of the operation mode:

- Writing a '0' to the CEN bit in the SRR register.
- Writing a '1' to the SRST bit in the SRR register. The core enters the Configuration mode immediately following the software reset.
- Driving a '0' on the S_AXI_ARESET_N input. The core continues to be in reset as long as S_AXI_ARESET_N is '0'. The core enters Configuration mode after S_AXI_ARESET_N is negated to '1'.

The following describes the Configuration mode features.

- CAN controller loses synchronization with the CAN bus and drives a constant recessive bit on the bus line.
- ECR register is reset.
- ESR register is reset.
- BTR and BRPR registers can be modified.
- CONFIG bit in the Status Register is '1.'
- CAN controller does not receive any new messages.
- CAN controller does not transmit any messages. Messages in the TX FIFO and the TX high priority buffer are kept pending. These packets are sent when normal operation is resumed.
- Reads from the RX FIFO can be performed.
- Writes to the TX FIFO and TX HPB can be performed.
- Interrupt Status Register bits ARBLST, TXOK, RXOK, RXOFLW, ERROR, BSOFF, SLP and WKUP are cleared.
- Interrupt Status Register bits RXNEMP, RXUFLW can be set due to read operations to the RX FIFO.
- Interrupt Status Register bits TXBFLL and TXFLL, and the Status Register bits TXBFLL and TXFLL, can be set due to write operations to the TX HPB and TX FIFO, respectively.
- Interrupts are generated if the corresponding bits in the IER are '1.'
- All Configuration Registers are accessible.

When in Configuration mode, the CAN controller continues to stay in this mode until the CEN bit in the SRR register is set to '1'. After the CEN bit is set to '1', the CAN controller waits for a sequence of 11 recessive bits before exiting Configuration mode.

The CAN controller enters Normal, Loop Back, or Sleep modes from Configuration mode, depending on the LBACK and SLEEP bits in the MSR Register.

Normal Mode

In Normal mode, the CAN controller participates in bus communication by transmitting and receiving messages. From Normal mode, the CAN controller can enter either Configuration or Sleep modes.

For Normal mode, the CAN controller state transitions are as follows:

- Enters Configuration mode when any configuration condition is satisfied
- Enters Sleep mode when the SLEEP bit in the MSR is '1'
- Enters Normal mode from Configuration mode only when the LBACK and SLEEP bits in the MSR are '0' and the CEN bit is '1'
- Enters Normal mode from Sleep mode when a wake-up condition occurs

Sleep Mode

The CAN controller enters Sleep mode from Configuration mode when the LBACK bit in MSR is '0', the SLEEP bit in MSR is '1', and the CEN bit in SRR is '1'. The CAN controller enters Sleep mode only when there are no pending transmission requests from either the TX FIFO or the TX High Priority Buffer.

The CAN controller enters Sleep mode from Normal mode only when the SLEEP bit is '1', the CAN bus is idle, and there are no pending transmission requests from either the TX FIFO or TX High Priority Buffer.

When another node transmits a message, the CAN controller receives the transmitted message and exits Sleep mode. When the controller is in Sleep mode, if there are new transmission requests from either the TX FIFO or the TX High Priority Buffer, these requests are serviced, and the CAN controller exits Sleep mode. Interrupts are generated when the CAN controller enters Sleep mode or wakes up from Sleep mode.

The CAN controller can enter either Configuration or Normal modes from Sleep mode.

The CAN controller can enter Configuration mode when any configuration condition is satisfied. The CAN controller enters Normal mode upon the following conditions (wake-up conditions):

- Whenever the SLEEP bit is set to '0'
- Whenever the SLEEP bit is '1', and bus activity is detected
- Whenever there is a new message in the TX FIFO or the TX High Priority Buffer

Loop Back Mode

In Loop Back mode, the CAN controller transmits a recessive bitstream on to the CAN Bus. Any message that is transmitted is looped back to the RX line and is acknowledged. The CAN controller receives any message that it transmits. It does not participate in normal bus communication and does not receive any messages that are transmitted by other CAN nodes.

This mode is used for diagnostic purposes. When in Loop Back mode, the CAN controller can enter Configuration mode only. The CAN controller enters Configuration mode when any of the configuration conditions are satisfied.

The CAN controller enters Loop Back mode from the Configuration mode if the LBACK bit in MSR is '1' and the CEN bit in SRR is '1.'

Clocking and Reset

Clocking

The CAN core has two clocks: CAN_CLK and S_AXI_ACLK. The following conditions apply for clock frequencies:

- CAN_CLK can be 8 to 24 MHz in frequency.
- CAN_CLK and S_AXI_ACLK can be asynchronous or can be clocked from the same source

Either of these clocks can be sourced from external oscillator sources or generated within the FPGA. The oscillator used for CAN_CLK must be compliant with the oscillator tolerance range given in the ISO 11898 -1, CAN 2.0A, and CAN 2.0B standards.

S_AXI_ACLK

The user can specify the operating frequency for S_AXI_ACLK. Using a DCM to generate this clock is optional. S_AXI_ACLK frequency must be greater than or equal to CAN_CLK frequency.

CAN_CLK

The range of CAN_CLK clock is 8-24 MHz.

The user determines whether a DCM or an external oscillator is used to generate the CAN_CLK. If an external oscillator is used, it should meet the tolerance requirements specified in the *ISO 11898-1*, *CAN 2.0A* and *CAN 2.0B* standards.

Reset Mechanism

Two different reset mechanisms are provided for the CAN controller. The S_AXI_ARESET_N input mentioned in [Table 1](#) acts as the system reset. Apart from the system reset, a software reset is provided through the SRST bit in the SRR register. Both the software reset and the system reset, reset the complete CAN core (the Object Layer and the Transfer Layer as shown in [Figure 1](#)).

Software Reset

The software reset can be enabled by writing a '1' to the SRST bit in the SRR Register. When a software reset is asserted, all the configuration registers including the SRST bit in the SRR Register are reset to their default values. Read/Write transactions can be performed starting at the next valid transaction window.

System Reset

The system reset can be enabled by driving a '0' on the S_AXI_ARESET_N input. All the configuration registers are reset to their default values. Read/Write transactions cannot be performed when the S_AXI_ARESET_N input is '0'.

Exceptions

The contents of the acceptance filter mask registers and acceptance filter ID registers are not cleared when the software reset or system reset is asserted.

Reset Synchronization

A reset synchronizer resets each clock domain in the core. Because of this, some latency exists between the assertion of reset and the actual reset of the core.

Interrupts

The CAN IP core uses a hard-vector interrupt mechanism. It has a single interrupt line (`IP2Bus_IntrEvent`) to indicate an interrupt. Interrupts are indicated by asserting the `IP2Bus_IntrEvent` line (transition of the `IP2Bus_IntrEvent` line from a logic '0' to a logic '1').

Events such as errors on the bus line, message transmission and reception, FIFO overflows and underflow conditions can generate interrupts. During power on, the Interrupt line is driven low.

The Interrupt Status Register (ISR) indicates the interrupt status bits. These bits are set and cleared regardless of the status of the corresponding bit in the Interrupt Enable Register (IER). The IER handles the interrupt-enable functionality. The clearing of a status bit in the ISR is handled by writing a '1' to the corresponding bit in the Interrupt Clear Register (ICR).

The following two conditions cause the `IP2Bus_IntrEvent` line to be asserted:

- If a bit in the ISR is '1' and the corresponding bit in the IER is '1'.
- Changing an IER bit from a '0' to '1'; when the corresponding bit in the ISR is already '1'.
- Two conditions cause the `IP2Bus_IntrEvent` line to be deasserted:
- Clearing a bit in the ISR that is '1' (by writing a '1' to the corresponding bit in the ICR); provided the corresponding bit in the IER is '1'.
- Changing an IER bit from '1' to '0'; when the corresponding bit in the ISR is '1'.

When both deassertion and assertion conditions occur simultaneously, the `IP2Bus_IntrEvent` line is deasserted first, and is reasserted if the assertion condition remains true.

Configuration Register Descriptions

Table 6 lists the CAN controller configuration registers. Each register is 32-bits wide and is represented in big endian format. Any read operations to reserved bits or bits that are not used return '0'. '0's are written to reserved bits and bit fields that are not used. Writes to reserved locations are ignored

Table 6: Configuration Registers

Register Name	AXI Address (offset from C_S_AXI_BASEADDR)	Access
Control Registers		
Software Reset Register (SRR)	0x000	Read/Write
Mode Select Register (MSR)	0x004	Read/Write
Transfer Layer Configuration Registers		
Baud Rate Prescaler Register (BRPR)	0x008	Read/Write
Bit Timing Register (BTR)	0x00C	Read/Write
Error Indication Registers		
Error Counter Register (ECR)	0x010	Read
Error Status Register (ESR)	0x014	Read/Write to Clear
CAN Status Registers		
Status Register (SR)	0x018	Read
Interrupt Registers		
Interrupt Status Register (ISR)	0x01C	Read
Interrupt Enable Register (IER)	0x020	Read/Write
Interrupt Clear Register (ICR)	0x024	Write
Reserved		
Reserved Locations	0x028 to 0x02C	Reads Return 0/ Write has no affect
Messages		
Transmit Message FIFO (TX FIFO)		
ID	0x030	Write
DLC	0x034	Write
Data Word 1	0x038	Write
Data Word 2	0x03C	Write
Transmit High Priority Buffer (TX HPB)		
ID	0x040	Write
DLC	0x044	Write
Data Word 1	0x048	Write
Data Word 2	0x04C	Write

Table 6: Configuration Registers

Receive Message FIFO (RX FIFO)			
	ID	0x050	Read
	DLC	0x054	Read
	Data Word 1	0x058	Read
	Data Word 2	0x05C	Read
Acceptance Filtering			
Acceptance Filter Register (AFR)		0x060	Read/Write
Acceptance Filter Mask Register 1 (AFMR1)		0x064	Read/Write
Acceptance Filter ID Register 1 (AFIR1)		0x068	Read/Write
Acceptance Filter Mask Register 2 (AFMR2)		0x06C	Read/Write
Acceptance Filter ID Register 2 (AFIR2)		0x070	Read/Write
Acceptance Filter Mask Register 3 (AFMR3)		0x074	Read/Write
Acceptance Filter ID Register 3 (AFIR3)		0x078	Read/Write
Acceptance Filter Mask Register 4 (AFMR4)		0x07C	Read/Write
Acceptance Filter ID Register 4 (AFIR4)		0x080	Read/Write
Reserved			
Reserved Locations		0x084 to 0x0FC	Reads Return 0/ Write has no affect

Control Registers

Software Reset Register

Writing '1' to the Software Reset Register (SRR) places the CAN controller in Configuration mode. When in Configuration mode, the CAN controller drives recessive on the bus line and does not transmit or receive messages. During power-up, the CEN and SRST bits are '0' and the CONFIG bit in the Status Register (SR) is '1'. The Transfer Layer Configuration Registers can be changed only when the CEN bit in the SRR Register is '0.'

Use these steps to configure the CAN controller at power up:

1. Configure the Transfer Layer Configuration Registers (BRPR and BTR) with the values calculated for the specific bit rate.

See [Baud Rate Prescaler Register](#) and [Bit Timing Register](#) for more information.

2. Do one of the following:
 - a. For Loop Back mode, write '1' to the LBACK bit in the MSR.
 - a. For Sleep mode, write '1' to the SLEEP bit in the MSR.

See [Operational Modes](#), page 8 for information about operational modes.

3. Set the CEN bit in the SRR to 1.

After the occurrence of 11 consecutive recessive bits, the CAN controller clears the CONFIG bit in the Status Register to '0', and sets the appropriate Status bit in the Status Register.

[Table 7](#) shows the bit positions in the SR register and [Table 8](#) provides the Software Reset Register bit descriptions.

Table 7: Software Reset Register Bit Position

0 — 29	30	31
Reserved	CEN	SRST

Table 8: Software Reset Register Bit Description

Bit(s)	Name	Core Access	Default Value	Description
0–29	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
30	CEN	Read/Write	0	CAN Enable: The Enable bit for the CAN controller. '1' = The CAN controller is in Loop Back, Sleep, or Normal mode depending on the LBACK and SLEEP bits in the MSR. '0' = The CAN controller is in the Configuration mode.
31	SRST	Read/Write	0	Reset: The Software reset bit for the CAN controller. '1' = CAN controller is reset. If a '1' is written to this bit, all the CAN controller configuration registers (including the SRR) are reset. Reads to this bit always return a '0.'

Mode Select Register

Writing to the Mode Select Register (MSR) enables the CAN controller to enter Sleep, Loop Back, or Normal modes. In Normal mode, the CAN controller participates in normal bus communication. If the SLEEP bit is set to '1', the CAN controller enters Sleep mode. If the LBACK bit is set to '1', the CAN controller enters Loop Back mode.

The LBACK and SLEEP bits should never both be '1' at the same time. At any given point the CAN controller can be either in Loop Back mode or Sleep mode, but not at the same time. If both are set, the LBACK Mode takes priority.

Table 9 shows the bit positions in the MSR and Table 10 provides MSR bit descriptions

Table 9: Model Select Register Bit Positions

0 — 29	30	31
Reserved	LBACK	SLEEP

Table 10: Model Select Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–29	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
30	LBACK	Read/Write	0	Loop Back Mode Select: The Loop Back Mode Select bit. '1' = CAN controller is in Loop Back mode. '0' = CAN controller is in Normal, Configuration, or Sleep mode. This bit can be written to only when the CEN bit in SRR is '0'.
31	SLEEP	Read/Write	0	Sleep Mode Select: The Sleep Mode select bit. '1' = CAN controller is in Sleep mode. '0' = CAN controller is in Normal, Configuration or Loop Back mode. This bit is cleared when the CAN controller wakes up from the Sleep mode.

Transfer Layer Configuration Registers

There are two Transfer Layer Configuration Registers: Baud Rate Prescaler Register (BRPR) and Bit Timing Register (BTR). These registers can be written to only when the CEN bit in the SRR is '0.'

Baud Rate Prescaler Register

The CAN clock for the CAN controller is divided by (prescaler + 1) to generate the quantum clock needed for sampling and synchronization. Table 11 shows the bit positions in the BRPR and Table 12 provides BRPR descriptions.

Table 11: Baud Rate Prescaler Register Positions

0 — 23	24 — 31
Reserved	BRP [7:0]

Table 12: Baud Rate Prescaler Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–23	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
24–31	BRP[7:0]	Read/Write	0	Baud Rate Prescaler: These bits indicate the prescaler value. The actual value ranges from 1–256.

The BRPR can be programmed to any value in the range 0–255. The actual value is one more than the value written into the register.

The CAN quantum clock can be calculated using this equation:

$$t_q = t_{osc} * (BRP + 1)$$

where t_q and t_{osc} are the time periods of the quantum and oscillator/system clocks respectively.

Note: A given CAN bit rate can be achieved with several bit-time configurations, but values should be chosen after careful consideration of oscillator tolerances and CAN propagation delays. For more information on CAN bit-time register settings, see the specifications *CAN 2.0A*, *CAN 2.0B*, and *ISO 11898-1*.

Bit Timing Register

The Bit Timing Register (BTR) specifies the bits needed to configure bit time. Specifically, the Propagation Segment, Phase segment 1, Phase segment 2, and Synchronization Jump Width (as defined in *CAN 2.0A*, *CAN 2.0B*, and *ISO 11898-1*) are written to the BTR. The actual value of each of these fields is one more than the value written to this register. [Table 13](#) shows the bit positions in the BTR and [Table 14](#) provides BTR bit descriptions.

Table 13: Bit Timing Register Bit Positions

0—22	23—24	25—27	28—31
Reserved	SJW[1..0]	TS2[2..0]	TS1[3..0]

Table 14: Bit Timing Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–22	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion
23–24	SJW[1..0]	Read/Write	0	Synchronization Jump Width: Indicates the Synchronization Jump Width as specified in the CAN 2.0A and CAN 2.0B standard. The actual value is one more than the value written to the register.
25–27	TS2[2..0]	Read/Write	0	Time Segment 2: Indicates Phase Segment 2 as specified in the CAN 2.0A and CAN 2.0B standard. The actual value is one more than the value written to the register.
28–31	TS1[3..0]	Read/Write	0	Time Segment 1: Indicates the Sum of Propagation Segment and Phase Segment 1 as specified in the CAN 2.0A and CAN 2.0B standard. The actual value is one more than the value written to the register.

The following equations can be used to calculate the number of time quanta in bit-time segments:

$$tTSEG1 = tq * (8 * TSEG1[3] + 4 * TSEG1[2] + 2 * TSEG1[1] + TSEG1[0] + 1)$$

$$tTSEG2 = tq * (4 * TSEG2[2] + 2 * TSEG2[1] + TSEG2[0] + 1)$$

$$tSJW = tq * (2 * SJW[1] + SJW[0] + 1)$$

where $tTSEG1$, $tTSEG2$, and $tSJW$ are the lengths of TS1, TS2, and SJW.

Note: A given bit-rate can be achieved with several bit-time configurations, but values should be chosen after careful consideration of oscillator tolerances and CAN propagation delays. For more information on CAN bit-time register settings, see the *CAN 2.0A*, *CAN 2.0B*, and *ISO 11898-1* specifications.

Error Indication Registers

The Error Counter Register (ECR) and the Error Status Register (ESR) comprise the Error Indication Registers.

Error Counter Register

The ECR is a read-only register. Writes to the ECR have no effect. The value of the error counters in the register reflect the values of the transmit and receive error counters in the CAN Protocol Engine Module (see [Figure 2](#)).

The following conditions reset the Transmit and Receive Error counters:

- When a '1' is written to the SRST bit in the SRR.
- When a '0' is written to the CEN bit in the SRR.
- When the CAN controller enters Bus Off state.
- During Bus Off recovery when the CAN controller enters Error Active state after 128 occurrences of 11 consecutive recessive bits.

When in Bus Off recovery, the Receive Error counter is advanced by 1 whenever a sequence of 11 consecutive recessive bits is seen.

[Table 15](#) shows the bit positions in the ECR and [Table 16](#) provides ECR bit descriptions.

Table 15: Error Count Register BIT Positions

0 — 15	16 — 23	24 — 31
Reserved	REC[7..0]	TEC[7..0]

Table 16: Error Count Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–15	Reserved	Read Only	0	Reserved: These bit positions are reserved for future expansion.
16–23	REC[7..0]	Read Only	0	Receive Error Counter: Indicates the value of the Receive Error Counter
24–31	TEC[7..0]	Read Only	0	Transmit Error Counter: Indicates the value of the Transmit Error Counter

Error Status Register

The Error Status Register (ESR) indicates the type of error that has occurred on the bus. If more than one error occurs, all relevant error flag bits are set in this register. The ESR is a write-to-clear register. Writes to this register does not set any bits, but does clear the bits that are set.

Table 17 shows the bit positions in the ESR and Table 18 provides ESR bit descriptions. All the bits in the ESR are cleared when a '0' is written to the CEN bit in the SRR.

Table 17: Error Status Register BIT Positions

0 — 26	27	28	29	30	31
Reserved	ACKER	BERR	STER	FMER	CRCER

Table 18: Error Status Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0—26	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
27	ACKER	Write to Clear	0	ACK Error: Indicates an acknowledgement error. '1' = Indicates that an acknowledgement error has occurred. '0' = Indicates that an acknowledgement error has not occurred on the bus since the last write to this register. If this bit is set, writing a '1' clears it.
28	BERR	Write to Clear	0	Bit Error: Indicates that the received bit is not the same as the transmitted bit during bus communication. '1' = Indicates that a bit error has occurred. '0' = Indicates that a bit error has not occurred on the bus since the last write to this register. If this bit is set, writing a '1' clears it.
29 ⁽¹⁾	STER	Write to Clear	0	Stuff Error: Indicates an error if there is a stuffing violation. '1' = Indicates that a stuff error has occurred. '0' = Indicates that a stuff error has not occurred on the bus since the last write to this register. If this bit is set, writing a '1' clears it.

1. In case of a CRC Error and a CRC delimiter corruption, only the FMER bit is set.

Status Register

The CAN Status Register provides a status of all conditions of the core. Specifically, FIFO status, Error State, Bus State and Configuration mode are reported.

Status Register

Table 19 shows the SR bit positions in the SR and Table 20 provides SR bit descriptions.

Table 19: Status Register BIT Positions

0 — 19	20	21	22	23 — 24	25
Reserved	ACFBSY	TXFLL	TXBFLL	ESTAT[1..0]	ERRWRN
26	27	28	29	30	31
BBSY	BIDLE	NORMAL	SLEEP	LBACK	CONFIG

Table 20: Status Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0—19	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
20	ACFBSY	Read Only	0	Acceptance Filter Busy: This bit indicates that the Acceptance Filter Mask Registers and the Acceptance Filter ID Registers cannot be written to. '1' = Acceptance Filter Mask Registers and Acceptance Filter ID Registers cannot be written to. '0' = Acceptance Filter Mask Registers and the Acceptance Filter ID Registers can be written to. This bit exists only when the number of acceptance filters is not '0' This bit is set when a '0' is written to any of the valid UAF bits in the Acceptance Filter Register.
21	TXFLL	Read Only	0	Transmit FIFO Full: Indicates that the TX FIFO is full. '1' = Indicates that the TX FIFO is full. '0' = Indicates that the TX FIFO is not full.
22	TXBFL	Read Only	0	High Priority Transmit Buffer Full: Indicates that the High Priority Transmit Buffer is full. '1' = Indicates that the High Priority Transmit Buffer is full. '0' = Indicates that the High Priority Transmit Buffer is not full.
28	NORMAL	Read Only	0	Normal Mode: Indicates that the CAN controller is in Normal Mode. '1' = Indicates that the CAN controller is in Normal Mode. '0' = Indicates that the CAN controller is not in Normal mode.
29	SLEEP	Read Only	0	Sleep Mode: Indicates that the CAN controller is in Sleep mode. '1' = Indicates that the CAN controller is in Sleep mode. '0' = Indicates that the CAN controller is not in Sleep mode.
30	LBACK	Read Only	0	Loop Back Mode: Indicates that the CAN controller is in Loop Back mode. '1' = Indicates that the CAN controller is in Loop Back mode. '0' = Indicates that the CAN controller is not in Loop Back mode.
31	CONFIG	Read Only	1	Configuration Mode Indicator: Indicates that the CAN controller is in Configuration mode. '1' = Indicates that the CAN controller is in Configuration mode. '0' = Indicates that the CAN controller is not in Configuration mode.

Interrupt Registers

The CAN controller contains a single interrupt line only, but contains several interrupt conditions. Interrupts are controlled by the interrupt status, enable, and clear registers.

Interrupt Status Register

The Interrupt Status Register (ISR) contains bits that are set when a specific interrupt condition occurs. If the corresponding mask bit in the Interrupt Enable Register is set, an interrupt is generated.

Interrupt bits in the ISR can be cleared by writing to the Interrupt Clear Register. For all bits in the ISR, a set condition takes priority over the clear condition and the bit continues to remain '1'.

Table 21 shows the bit positions in the ISR and Table 22 provides ISR descriptions.

Table 21: Interrupt Status Register Bit Positions

0 — 19	20	21	22	23	24	25
Reserved	WKUP	SLP	BSOFF	ERROR	RXNEMP	RXOFLW
26	27	28	29	30	31	
RXUFLW	RXOK	TXBFLL	TXFLL	TXOK	ARBLST	

Table 22: Interrupt Status Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–19	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
20	WKUP	Read Only	0	Wake up Interrupt: A '1' indicates that the CAN controller entered Normal mode from Sleep Mode. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
21	SLP	Read Only	0	Sleep Interrupt: A '1' indicates that the CAN controller entered Sleep mode. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
22	BSOFF	Read Only	0	Bus Off Interrupt: A '1' indicates that the CAN controller entered the Bus Off state. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
23	ERROR	Read Only	0	Error Interrupt: A '1' indicates that an error occurred during message transmission or reception. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
24	RXNEMP	Read Only	0	Receive FIFO Not Empty Interrupt: A '1' indicates that the Receive FIFO is not empty. This bit can be cleared only by writing to the ICR.
25	RXOFLW	Read Only	0	RX FIFO Overflow Interrupt: A '1' indicates that a message has been lost. This condition occurs when a new message is being received and the Receive FIFO is Full. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
26	RXUFLW	Read Only	0	RX FIFO Underflow Interrupt: A '1' indicates that a read operation was attempted on an empty RX FIFO. This bit can be cleared only by writing to the ICR.
27	RXOK	Read Only	0	New Message Received Interrupt: A '1' indicates that a message was received successfully and stored into the RX FIFO. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
28	TXBFLL	Read Only	0	High Priority Transmit Buffer Full Interrupt: A '1' indicates that the High Priority Transmit Buffer is full. The status of the bit is unaffected if write transactions occur on the High Priority Transmit Buffer when it is already full. This bit can be cleared only by writing to the ICR.

Table 22: Interrupt Status Register Bit Descriptions (Cont'd)

29	TXFLL	Read Only	0	Transmit FIFO Full Interrupt: A '1' indicates that the TX FIFO is full. The status of the bit is unaffected if write transactions occur on the Transmit FIFO when it is already full. This bit can be cleared only by writing to the Interrupt Clear Register.
30	TXOK ⁽¹⁾	Read Only	0	Transmission Successful Interrupt: A '1' indicates that a message was transmitted successfully. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.
31	ARBLST	Read Only	0	Arbitration Lost Interrupt: A '1' indicates that arbitration was lost during message transmission. This bit can be cleared by writing to the ICR or when '0' is written to the CEN bit in the SRR.

1. In Loop Back mode, both TXOK and RXOK bits are set. The RXOK bit is set before the TXOK bit.

Interrupt Enable Register

The Interrupt Enable Register (IER) is used to enable interrupt generation. Table 23 shows the bit positions in the IER and Table 24 provides IER bit descriptions.

Table 23: Interrupt Enable Register Bit Positions

0 — 19	20	21	22	23	24	25
Reserved	EWKUP	ESLP	EBSOFF	EERROR	ERXNEMP	ERXOFLW
26	27	28	29	30	31	
ERXUFLW	ERXOK	ETXBFLW	ETXFLL	ETXOK	EARBLST	

Table 24: Interrupt Enable Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–19	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
20	EWKUP	Read/Write	0	Enable Wake up Interrupt: Writes to this bit enable or disable interrupts when the WKUP bit in the ISR is set. '1' = Enable interrupt generation if WKUP bit in ISR is set. '0' = Disable interrupt generation if WKUP bit in ISR is set.
21	ESLP	Read/Write	0	Enable Sleep Interrupt: Writes to this bit enable or disable interrupts when the SLP bit in the ISR is set. '1' = Enable interrupt generation if SLP bit in ISR is set. '0' = Disable interrupt generation if SLP bit in ISR is set.
22	EBSOFF	Read/Write	0	Enable Bus OFF Interrupt: Writes to this bit enable or disable interrupts when the BSOFF bit in the ISR is set. '1' = Enable interrupt generation if BSOFF bit in ISR is set. '0' = Disable interrupt generation if BSOFF bit in ISR is set.
23	EERROR	Read/Write	0	Enable Error Interrupt: Writes to this bit enable or disable interrupts when the ERROR bit in the ISR is set. '1' = Enable interrupt generation if ERROR bit in ISR is set. '0' = Disable interrupt generation if ERROR bit in ISR is set.

Table 24: Interrupt Enable Register Bit Descriptions (Cont'd)

24	ERXNEMP	Read/Write	0	Enable Receive FIFO Not Empty Interrupt: Writes to this bit enable or disable interrupts when the RXNEMP bit in the ISR is set. '1' = Enable interrupt generation if RXNEMP bit in ISR is set. '0' = Disable interrupt generation if RXNEMP bit in ISR is set.
25	ERXOFLW	Read/Write	0	Enable RX FIFO Overflow Interrupt: Writes to this bit enable or disable interrupts when the RXOFLW bit in the ISR is set. '1' = Enable interrupt generation if RXOFLW bit in ISR is set. '0' = Disable interrupt generation if RXOFLW bit in ISR is set.
26	ERXUFLW	Read/Write	0	Enable RX FIFO Underflow Interrupt: Writes to this bit enable or disable interrupts when the RXUFLW bit in the ISR is set. '1' = Enable interrupt generation if RXUFLW bit in ISR is set. '0' = Disable interrupt generation if RXUFLW bit in ISR is set.
27	ERXOK	Read/Write	0	Enable New Message Received Interrupt: Writes to this bit enable or disable interrupts when the RXOK bit in the ISR is set. '1' = Enable interrupt generation if RXOK bit in ISR is set. '0' = Disable interrupt generation if RXOK bit in ISR is set.
28	ETXBFL	Read/Write	0	Enable High Priority Transmit Buffer Full Interrupt: Writes to this bit enable or disable interrupts when the TXBFL bit in the ISR is set. '1' = Enable interrupt generation if TXBFL bit in ISR is set. '0' = Disable interrupt generation if TXBFL bit in ISR is set.
29	ETXFLL	Read/Write	0	Enable Transmit FIFO Full Interrupt: Writes to this bit enable or disable interrupts when the TXFLL bit in the ISR is set. '1' = Enable interrupt generation if TXFLL bit in ISR is set. '0' = Disable interrupt generation if TXFLL bit in ISR is set.
30	ETXOK	Read/Write	0	Enable Transmission Successful Interrupt: Writes to this bit enable or disable interrupts when the TXOK bit in the ISR is set. '1' = Enable interrupt generation if TXOK bit in ISR is set. '0' = Disable interrupt generation if TXOK bit in ISR is set.
31	EARBLST	Read/Write	0	Enable Arbitration Lost Interrupt: Writes to this bit enable or disable interrupts when the ARBLST bit in the ISR is set. '1' = Enable interrupt generation if ARBLST bit in ISR is set. '0' = Disable interrupt generation if ARBLST bit in ISR is set.

Interrupt Clear Register

The Interrupt Clear Register (ICR) is used to clear interrupt status bits. Table 25 shows the bit positions in the ICR and Table 26 gives the ICR bit descriptions.

Table 25: Interrupt Clear Register Bit Positions

0 — 19	20	21	22	23	24	25
Reserved	CWKUP	CSLP	CBSOFF	CERROR	CRXNEMP	CRXOFLW
26	27	28	29	30	31	
CRXUFLW	CRXOK	CTXBFL	CTXFLL	CTXOK	CARBLST	

Table 26: Interrupt Clear Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–19	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
20	CWKUP	Write Only	0	Clear Wake up Interrupt: Writing a ‘1’ to this bit clears the WKUP bit in the ISR.
21	CSLP	Write Only	0	Clear Sleep Interrupt: Writing a ‘1’ to this bit clears the SLP bit in the ISR.
22	CBSOFF	Write Only	0	Clear Bus Off Interrupt: Writing a ‘1’ to this bit clears the BSOFF bit in the ISR.
23	CERROR	Write Only	0	Clear Error Interrupt: Writing a ‘1’ to this bit clears the ERROR bit in the ISR.
24	CRXNEMP	Write Only	0	Clear Receive FIFO Not Empty Interrupt: Writing a ‘1’ to this bit clears the RXNEMP bit in the ISR.
25	CRXOFLW	Write Only	0	Clear RX FIFO Overflow Interrupt: Writing a ‘1’ to this bit clears the RXOFLW bit in the ISR.
26	CRXUFLW	Write Only	0	Clear RX FIFO Underflow Interrupt: Writing a ‘1’ to this bit clears the RXUFLW bit in the ISR.
27	CRXOK	Write Only	0	Clear New Message Received Interrupt: Writing a ‘1’ to this bit clears the RXOK bit in the ISR.
28	CTXBFLL	Write Only	0	Clear High Priority Transmit Buffer Full Interrupt: Writing a ‘1’ to this bit clears the TXBFLL bit in the ISR.
29	CTXFLL	Write Only	0	Clear Transmit FIFO Full Interrupt: Writing a ‘1’ to this bit clears the TXFLL bit in the ISR.
30	CTXOK	Write Only	0	Clear Transmission Successful Interrupt: Writing a ‘1’ to this bit clears the TXOK bit in the ISR.
31	CARBLST	Write Only	0	Clear Arbitration Lost Interrupt: Writing a ‘1’ to this bit clears the ARBLST bit in the ISR.

Message Storage

The CAN controller has a Receive FIFO (RX FIFO) for storing received messages. The RX FIFO depth is configurable and can store up to 64 messages.

Messages that pass any of the acceptance filters are stored in the RX FIFO. When no acceptance filter has been selected, all received messages are stored in the RX FIFO.

The CAN controller has a configurable Transmit FIFO (TX FIFO) that can store up to 64 messages. The CAN controller also has a High Priority Transmit Buffer (TX HPB), with storage for one message. When a higher priority message needs to be sent, write the message to the High Priority Transmit Buffer. The message in the Transmit Buffer has priority over messages in the TX FIFO.

Message Transmission and Reception

The following rules apply regarding message transmission and reception:

- A message in the TX High Priority Buffer (TX HPB) has priority over messages in the TX FIFO.
- In case of arbitration loss or errors during the transmission of a message, the CAN controller tries to retransmit the message. No subsequent message, even a newer, higher priority message is transmitted until the original message is transmitted without errors or arbitration loss.
- The messages in the TX FIFO, TX HPB and RX FIFO are retained even if the CAN controller enters Bus off state or Configuration mode.

Message Structure

Each message is 16 bytes. Byte ordering for the CAN message structure is shown in [Table 27](#), [Table 28](#), [Table 29](#), and [Table 30](#).

Table 27: Message Identifier [IDR]

0 — 10	11	12	13 — 30	31
ID [28..18]	SRR/RTR	IDE	ID[17..0]	RTR

Table 28: Data Length Code [DLCR]

0 — 3	4 — 31
DLC [3..0]	Reserved

Table 29: Data Word 1 [DW1R]

0 — 7	8 — 15	16 — 23	24 — 31
DB0[7..0]	DB1[7..0]	DB2[7..0]	DB3[7..0]

Table 30: Data Word 2 [DW2R]

0 — 7	8 — 15	16 — 23	24 — 31
DB4[7..0]	DB5[7..0]	DB6[7..0]	DB7[7..0]

All 16 bytes must be read from the RX FIFO to receive the complete message. The first word read (4 bytes) returns the identifier of the received message (IDR). The second read returns the Data Length Code (DLC) field of the received message (DLCR). The third read returns Data Word 1 (DW1R), and the fourth read returns Data Word 2 (DW2R).

All four words have to be read for each message, even if the message contains less than 8 data bytes. Write transactions to the RX FIFO are ignored. Reads from an empty RX FIFO return invalid data.

Writes to TX FIFO and High Priority TX Buffer

When writing to the TX FIFO or the TX HPB, all 16 bytes must be written. The first word written (4 bytes) is the Identifier (IDR). The second word written is the DLC field (DLCR). The third word written is Data Word 1 (DW1R) and the fourth word written is Data Word 2 (DW2R).

When transmitting on the CAN bus, the CAN controller transmits the data bytes in the following order (DB0, DB1, DB2, DB3, DB4, DB5, DB6, DB7). The MSB of a data byte is transmitted first.

All four words must be written for each message, including messages containing fewer than 8 data bytes. Reads transactions from the TX FIFO or the TX High Priority Buffer return '0's.

- '0's must be written to Data Fields in DW1R and DW2R registers that are not used
- '0's must be written to bits 4 to 31 in the DLCR
- '0's must be written to IDR [13 to 31] for standard frames

Identifier

The Identifier (IDR) word contains the identifier field of the CAN message. Two different formats exist for the Identifier field of the CAN message frame: Standard and Extended frames.

- **Standard Frames:** Standard frames have an 11-bit identifier field called the Standard Identifier. Only the ID[28..18], SRR/RTR, and IDE bits are valid. ID[28..18] is the 11 bit identifier. The SRR/RTR bit differentiates between data and remote frames. IDE is '0' for standard frames. The other bit fields are not used.
- **Extended Frames:** Extended frames have an 18-bit identifier extension in addition to the Standard Identifier. All bit fields are valid. The RTR bit is used to differentiate between data and remote frames (The SRR/RTR bit and IDE bit are both '1' for all Extended Frames).

[Table 31](#) provides bit descriptions for the Identifier Word. [Table 32](#) provides bit descriptions for the DLC Word. [Table 33](#) provides bit descriptions for Data Word 1 and Data Word 2

Table 31: Identifier Word Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–10	ID[28..18]	Reads from RX FIFO Writes to TX FIFO and TX HPB	0	Standard Message ID: The Identifier portion for a Standard Frame is 11 bits. These bits indicate the Standard Frame ID. This field is valid for both Standard and Extended Frames.
11	SRR/RTR	Reads from RX FIFO Writes to TX FIFO and TX HPB	0	Substitute Remote Transmission Request: This bit differentiates between data frames and remote frames. Valid only for Standard Frames. For Extended frames this bit is 1. ‘1’ = Indicates that the message frame is a Remote Frame. ‘0’ = Indicates that the message frame is a Data Frame.
12	IDE	Reads from RX FIFO Writes to TX FIFO and TX HPB	0	Identifier Extension: This bit differentiates between frames using the Standard Identifier and those using the Extended Identifier. Valid for both Standard and Extended Frames. ‘1’ = Indicates the use of an Extended Message Identifier. ‘0’ = Indicates the use of a Standard Message Identifier.
13–30	ID[17..0]	Reads from RX FIFO Writes to TX FIFO and TX HPB	0	Extended Message ID: This field indicates the Extended Identifier. Valid only for Extended Frames. For Standard Frames, reads from this field return ‘0’s For Standard Frames, writes to this field should be ‘0’s
31	RTR	Reads from RX FIFO Writes to TX FIFO and TX HPB	0	Remote Transmission Request: This bit differentiates between data frames and remote frames. Valid only for Extended Frames. ‘1’ = Indicates that the message object is a Remote Frame ‘0’ = Indicates that the message object is a Data Frame For Standard Frames, reads from this bit returns ‘0’ For Standard Frames, writes to this bit should be ‘0’

Table 32: DLC Word Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–3	DLC	Read/Write	0	Data Length Code: This is the data length portion of the control field of the CAN frame. This indicates the number valid data bytes in Data Word 1 and Data Word 2 registers.
4–31	Reserved	Read/Write		Reads from this field return ‘0’s. Writes to this field should be ‘0’s.

Table 33: Data Word 1 and Data Word 2 Bit Descriptions

Register	Field	Core Access	Default Value	Description
DW1R [0..7]	DB0[7..0]	Read/Write	0	Data Byte 0: Reads from this field return invalid data if the message has no data.
DW1R [8..15]	DB1[7..0]	Read/Write	0	Data Byte 1: Reads from this field return invalid data if the message has only 1 byte of data or fewer.
DW1R [16..23]	DB2[7..0]	Read/Write	0	Data Byte 2: Reads from this field return invalid data if the message has 2 bytes of data or fewer.
DW1R [24..31]	DB3[7..0]	Read/Write	0	Data Byte 3: Reads from this field return invalid data if the message has 3 bytes of data or fewer.
DW2R [0..7]	DB4[7..0]	Read/Write	0	Data Byte 4: Reads from this field return invalid data if the message has 4 bytes of data or fewer.
DW2R [8..15]	DB5[7..0]	Read/Write	0	Data Byte 5: Reads from this field return invalid data if the message has 5 bytes of data or fewer.
DW2R [16..23]	DB6[7..0]	Read/Write	0	Data Byte 6: Reads from this field return invalid data if the message has 6 bytes of data or fewer.
DW2R [24..31]	DB7[7..0]	Read/Write	0	Data Byte 7: Reads from this field return invalid data if the message has 7 bytes of data or fewer.

Acceptance Filters

The number of acceptance filters is configurable from 0 to 4. The parameter *Number of Acceptance Filters* specifies the number of acceptance filters that are chosen. Each acceptance filter has an Acceptance Filter Mask Register and an Acceptance Filter ID Register.

Acceptance filtering is performed in the following sequence:

1. The incoming Identifier is masked with the bits in the Acceptance Filter Mask Register.
2. The Acceptance Filter ID Register is also masked with the bits in the Acceptance Filter Mask Register.
3. The resultant values are compared.
4. If the values are equal, the message is stored in the RX FIFO.
5. Acceptance filtering is processed by each of the defined filters. If the incoming identifier passes through any acceptance filter, the message is stored in the RX FIFO.

The following rules apply to the Acceptance filtering process:

- If no acceptance filters are selected (for example, if all the valid UAF bits in the AFR register are '0's or if the parameter *Number of Acceptance Filters* = '0'), all received messages are stored in the RX FIFO.
- If the number of acceptance filters is greater than or equal to 1, all the Acceptance Filter Mask Register and the Acceptance Filter ID Register locations can be written to and read from. However, the use of these filter pairs for acceptance filtering is governed by the existence of the UAF bits in the AFR register.

Acceptance Filter Register

The Acceptance Filter Register (AFR) defines which acceptance filters to use. Each Acceptance Filter ID Register (AFIR) and Acceptance Filter Mask Register (AFMR) pair is associated with a UAF bit.

When the UAF bit is '1', the corresponding acceptance filter pair is used for acceptance filtering. When the UAF bit is '0', the corresponding acceptance filter pair is not used for acceptance filtering. The AFR exists only if the *Number of Acceptance Filters* parameter is not set to '0.'

To modify an acceptance filter pair in Normal mode, the corresponding UAF bit in this register must be set to '0.' After the acceptance filter is modified, the corresponding UAF bit must be set to '1.'

These conditions govern the number of UAF bits that can exist in the AFR:

- If the number of acceptance filters is 1:UAF1 bit exists
- If the number of acceptance filters is 2:UAF1 and UAF2 bits exist
- If the number of acceptance filters is 3:UAF1, UAF2 and UAF3 bits exist
- If the number of acceptance filters is 4:UAF1, UAF2, UAF3 and UAF4 bits exist
- UAF bits for filters that do not exist are not written to
- Reads from UAF bits that do not exist return '0's
- If all existing UAF bits are set to '0', all received messages are stored in the RX FIFO
- If the UAF bits are changed from a '1' to '0' during reception of a CAN message, the message may not be stored in the RX FIFO.

[Table 34](#) shows the bit positions in the AFR and [Table 35](#) gives the AFR bit descriptions.

Table 34: Acceptance Filter Register Bit Positions

0 — 27	28	29	30	31
Reserved	UAF4	UAF3	UAF2	UAF1

Table 35: Acceptance Filter Register Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–27	Reserved	Read/Write	0	Reserved: These bit positions are reserved for future expansion.
28	UAF4	Read/Write	0	Use Acceptance Filter Number 4: Enables the use of acceptance filter pair 4. '1' = Indicates that Acceptance Filter Mask Register 4 and Acceptance Filter ID Register 4 are used for acceptance filtering. '0' = Indicates that Acceptance Filter Mask Register 4 and Acceptance Filter ID Register 4 are not used for acceptance filtering.
29	UAF3	Read/Write	0	Use Acceptance Filter Number 3: Enables the use of acceptance filter pair 3. '1' = Indicates that Acceptance Filter Mask Register 3 and Acceptance Filter ID Register 3 are used for acceptance filtering. '0' = Indicates that Acceptance Filter Mask Register 3 and Acceptance Filter ID Register 3 are not used for acceptance filtering.
30	UAF2	Read/Write	0	Use Acceptance Filter Number 2: Enables the use of acceptance filter pair 2. '1' = Indicates that Acceptance Filter Mask Register 2 and Acceptance Filter ID Register 2 are used for acceptance filtering. '0' = Indicates that Acceptance Filter Mask Register 2 and Acceptance Filter ID Register 2 are not used for acceptance filtering.
31	UAF1	Read/Write	0	Use Acceptance Filter Number 1: Enables the use of acceptance filter pair 1. '1' = Indicates that Acceptance Filter Mask Register 1 and Acceptance Filter ID Register 1 are used for acceptance filtering. '0' = Indicates that Acceptance Filter Mask Register 1 and Acceptance Filter ID Register 1 are not used for acceptance filtering.

Acceptance Filter Mask Registers

The Acceptance Filter Mask Registers (AFMR) contain mask bits that are used for acceptance filtering. The incoming message identifier portion of a message frame is compared with the message identifier stored in the acceptance filter ID register. The mask bits define which identifier bits stored in the acceptance filter ID register are compared to the incoming message identifier.

There are at most four AFMRs. These registers are stored in a block RAM. Asserting a software reset or system reset does not clear register contents. If the number of acceptance filters is greater than or equal to 1, all four AFMRs are defined. These registers can be read from and written to. However, filtering operations are only performed on the number of filters defined by the *Number of Acceptance Filters* parameter. These registers are written to only when the corresponding UAF bits in the AFR are '0' and ACFBSY bit in the SR is '0'.

The following conditions govern AFMRs:

- If the number of acceptance filters is 1, AFMR 1 is used for acceptance filtering.
- If the number of acceptance filters is 2, AFMR 1 and AFMR 2 are used for acceptance filtering.
- If the number of acceptance filters is 3, AFMR 1, AFMR 2 and AFMR 3 are used for acceptance filtering.
- If the number of acceptance filters is 4, AFMR 1, AFMR 2, AFMR 3 and AFMR 4 are used for acceptance filtering.
- **Extended Frames:** All bit fields (AMID [28..18], AMSRR, AMIDE, AMID [17..0] and AMRTR) need to be defined.
- **Standard Frames:** Only AMID [28..18], AMSRR and AMIDE need to be defined. AMID [17..0] and AMRTR should be written as '0'.

[Table 36](#) shows the bit positions in the AFMR and [Table 37](#) provides bit descriptions.

Table 36: Acceptance Filter Mask Registers Bit Positions

0 — 10	11	12	13 — 30	31
AMID[28..18]	AMSRR	AMIDE	AMID[17..0]	AMRTR

Table 37: Acceptance Filter Mask Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–10	AMID [28..18]	Read/Write	0	Standard Message ID Mask: These bits are used for masking the Identifier in a Standard Frame. ‘1’= Indicates that the corresponding bit in the Acceptance Mask ID Register is used when comparing the incoming message identifier. ‘0’ = Indicates that the corresponding bit in the Acceptance Mask ID Register is not used when comparing the incoming message identifier.
11	AMSRR	Read/Write	0	Substitute Remote Transmission Request Mask: This bit is used for masking the RTR bit in a Standard Frame. ‘1’= Indicates that the corresponding bit in the Acceptance Mask ID Register is used when comparing the incoming message identifier. ‘0’ = Indicates that the corresponding bit in the Acceptance Mask ID Register is not used when comparing the incoming message identifier.
12	AMIDE	Read/Write	0	Identifier Extension Mask: Used for masking the IDE bit in CAN frames. ‘1’= Indicates that the corresponding bit in the Acceptance Mask ID Register is used when comparing the incoming message identifier. ‘0’ = Indicates that the corresponding bit in the Acceptance Mask ID Register is not used when comparing the incoming message identifier. If AMIDE = ‘1’ and the AIIDE bit in the corresponding Acceptance ID register is ‘0’, this mask is applicable to only Standard frames. If AMIDE = ‘1’ and the AIIDE bit in the corresponding Acceptance ID register is ‘1’, this mask is applicable to only extended frames. If AMIDE = ‘0’ this mask is applicable to both Standard and Extended frames.
13–30	AMID[17..0]	Read/Write	0	Extended Message ID Mask: These bits are used for masking the Identifier in an Extended Frame. ‘1’= Indicates that the corresponding bit in the Acceptance Mask ID Register is used when comparing the incoming message identifier. ‘0’ = Indicates that the corresponding bit in the Acceptance Mask ID Register is not used when comparing the incoming message identifier.
31	AMRTR	Read/Write	0	Remote Transmission Request Mask: This bit is used for masking the RTR bit in an Extended Frame. ‘1’= Indicates that the corresponding bit in the Acceptance Mask ID Register is used when comparing the incoming message identifier. ‘0’ = Indicates that the corresponding bit in the Acceptance Mask ID Register is not used when comparing the incoming message identifier.

Acceptance Filter ID Registers

The Acceptance Filter ID registers (AFIR) contain Identifier bits, which are used for acceptance filtering. There are at most four Acceptance Filter ID Registers. These registers are stored in a block RAM. Asserting a software reset or system reset does not clear the contents of these registers. If the number of acceptance filters is greater than or equal to 1, all four AFIRs are defined. These registers can be read from and written to. These registers should be written to only when the corresponding UAF bits in the AFR are '0' and ACFBSY bit in the SR is '0'.

The following conditions govern the use of the AFIRs:

- If the number of acceptance filters is 1, AFIR 1 is used for acceptance filtering.
- If the number of acceptance filters is 2, AFIR 1 and AFIR 2 are used for acceptance filtering.
- If the number of acceptance filters is 3, AFIR 1, AFIR 2 and AFIR 3 are used for acceptance filtering.
- If the number of acceptance filters is 4, AFIR 1, AFIR 2, AFIR 3 and AFIR 4 are used for acceptance filtering.
- **Extended Frames:** All the bit fields (AIID [28..18], AISRR, AIIDE, AIID [17..0] and AIRTR) must be defined.
- **Standard Frames:** Only AIID [28..18], AISRR and AIIDE need to be defined. AIID [17..0] and AIRTR should be written with '0'

Table 38 shows AFIR bit positions. Table 39 provides AFIR bit descriptions.

Table 38: Acceptance Filter ID Registers Bit Positions

0 — 10	11	12	13 — 30	31
AIID[28..18]	AISRR	AIIDE	AIID[17..0]	AIRTR

Table 39: Acceptance Filter ID Registers Bit Descriptions

Bit(s)	Name	Core Access	Default Value	Description
0–10	AIID [28..18]	Read/Write	0	Standard Message ID: This is the Standard Identifier.
11	AISRR	Read/Write	0	Substitute Remote Transmission Request: Indicates the Remote Transmission Request bit for Standard frames
12	AIIDE	Read/Write	0	Identifier Extension: Differentiates between Standard and Extended frames
13–30	AIID[17..0]	Read/Write	0	Extended Message ID: Extended Identifier
31	AIRTR	Read/Write	0	Remote Transmission Request Mask: RTR bit for Extended frames.

Configuring the CAN Controller

This section covers the various configuration steps that must be performed to program the CAN core for operation.

The following key configuration steps are detailed in this section.

1. Choose the mode of operation of the CAN core.
2. Program the configuration registers to initialize the core.
3. Write messages to the TX FIFO/ TX HPB.
4. Read messages from the RX FIFO.

Programming the Configuration Registers

The following are steps to configure the core when the core is powered-on, or after system or software reset.

1. Choose the operation mode.
 - a. For Loop Back mode, write a '1' to the LBACK bit in the MSR and '0' to the SLEEP bit in the MSR.
 - b. For Sleep mode, write a '1' to the SLEEP bit in the MSR and '0' to the LBACK bit in the MSR.
 - c. For Normal Mode, write '0's to the LBACK and SLEEP bits in the MSR.
2. Configure the Transfer Layer Configuration Registers.
 - a. Program the Baud Rate Prescaler Register and the Bit Timing Register to correspond to the network timing parameters and the network characteristics of the system.
3. Configure the Acceptance Filter Registers.

The number of Acceptance Filter Mask and Acceptance Filter ID Register pairs is chosen at build time. To configure these registers do the following:

 - a. Write a '0' to the UAF bit in the AFR register corresponding to the Acceptance Filter Mask and ID Register pair to be configured.
 - b. Wait until the ACFBSY bit in the SR is '0'.
 - c. Write the appropriate mask information to the Acceptance Filter Mask Register.
 - d. Write the appropriate ID information to the to the Acceptance Filter ID Register.
 - e. Write a '1' to the UAF bit corresponding to the Acceptance Filter Mask and ID Register pair.
 - f. Repeat the preceding steps for each Acceptance Filter Mask and ID Register pair.
4. Write to the Interrupt Enable Register to choose the bits in the Interrupt Status Register than can generate an interrupt.
5. Enable the CAN controller by writing a '1' to the CEN bit in the SRR register.

Transmitting a Message

A message to be transmitted can be written to either the TX FIFO or the TX HPB. A message in the TX HPB gets priority over the messages in the TX FIFO. The TXOK bit in the ISR is set after the CAN core successfully transmits a message.

Writing a Message to the TX FIFO

All messages written to the TX FIFO should follow the format described in [Message Storage, page 24](#).

To perform a write:

1. Poll the TXFLL bit in the SR. The message can be written into the TX FIFO when the TXFLL bit is '0'.
2. Write the ID of the message to the TX FIFO ID memory location (0x030).
3. Write the DLC of the message to the TX FIFO DLC memory location (0x034).
4. Write Data Word 1 of the message to the TX FIFO DW1 memory location (0x038).
5. Write Data Word 2 of the message to the TX FIFO DW2 memory location (0x03C).

Messages can be continuously written to the TX FIFO until the TX FIFO is full. When the TX FIFO is full, the TXFLL bit in the ISR and the TXFLL bit in the SR are set. If polling, the TXFLL bit in the Status Register should be polled after each write. If using interrupt mode, writes can continue until the TXFLL bit in the ISR generates an interrupt.

Writing a Message to the TX HPB

All messages written to the TX FIFO should follow the format described in [Message Storage, page 24](#).

To write a message to the TX HPB:

1. Poll the TXBFL bit in the SR.
The message can be written into the TX HPB when the TXBFL bit is '0'.
2. Write the ID of the message to the TX HPB ID memory location (0x040).
3. Write the DLC of the message to the TX HPB DLC memory location (0x044).
4. Write Data Word 1 of the message to the TX HPB DW1 memory location (0x048).
5. Write Data Word 2 of the message to the TX HPB DW2 memory location (0x04C).

After each write to the TX HPB, the TXBFL bit in the Status Register and the TXBFL bit in the Interrupt Status Register are set.

Receiving a Message

Whenever a new message is received and written into the RX FIFO, the RXNEMP bit and the RXOK bits in the ISR are set. In case of a read operation on an empty RX FIFO, the RXUFLW bit in the ISR is set.

Reading a Message from the RX FIFO

Perform the following steps to read a message from the RX FIFO.

1. Poll the RXOK or RXNEMP bits in the ISR. In interrupt mode, the reads can occur after the RXOK or RXNEMP bits in the ISR generate an interrupt.
 - a. Read from the RX FIFO memory locations. All the locations must be read regardless of the number of data bytes in the message.
 - b. Read from the RX FIFO ID location (0x050)
 - c. Read from the RX FIFO DLC location (0x054)
 - d. Read from the RX FIFO DW1 location (0x058)
 - e. Read from the RX FIFO DW2 location (0x05C)
2. After performing the read, if there are one or more messages in the RX FIFO, the RXNEMP bit in the ISR is set. This bit can either be polled or can generate an interrupt.
3. Repeat until the FIFO is empty.

Extra Design Consideration

The AXI CAN cores requires an input register on the RX line to avoid a potential error condition where multiple registers receive different values resulting in error frames. This error condition is rare; however, the work-around should be implemented in all cases. To work around this issue, insert a register on the RX line clocked by CAN_CLK with an initial value of '1'. This applies to all versions of the AXI CAN cores.

Design Implementation

Device and Package Selection

- The AXI CAN can be implemented in an FPGA listed in the Supported Device Family field in the LogiCORE IP Facts Table. Ensure that the device used has the following attributes:
- The device is large enough to accommodate the core, and
- It contains a sufficient number of IOBs.

Location Constraints

No specific I/O location constraints.

Placement Constraints

No specific placement constraints.

Timing Constraints

The core has two different clock domains: S_AXI_ACLK and CAN_CLK. The constraints given in the following sections can be used with the CAN Controller.

PERIOD Constraints for Clock Nets

CAN_CLK

The clock provided to CAN_CLK must be constrained for a clock frequency of less than or equal to 24 MHz, based on the input oscillator frequency.

```
# Set the CAN_CLK constraints
```

```
NET "CAN_CLK" TNM_NET = "CAN_CLK";
TIMESPEC "TS_CAN_CLK" = PERIOD "CAN_CLK" 40 ns HIGH 50%;
```

S_AXI_ACLK

The clock provided to S_AXI_ACLK must be constrained for a clock frequency of 100 MHz or less.

```
# Set the S_AXI_ACLK constraints
```

```
# This can be relaxed based on the actual frequency
```

```
NET "S_AXI_ACLK" TNM_NET = "S_AXI_ACLK";
TIMESPEC "TS_S_AXI_ACLK" = PERIOD "S_AXI_ACLK" 10 ns HIGH 50%;
```

Timing Ignore Constraints

For all the signals that cross clock domains, the following timing ignore (TIG) constraints are specified in the default UCF of the axi_can core. This default UCF is created in the implementation directory, under the core's wrapper files' directory.

```
# Timing Ignore constraint on all signals that cross from CAN_CLK domain to S_AXI_ACLK domain
```

```
TIMESPEC "TS_CAN_SYS_TIG" = FROM "CAN_CLK" TO "S_AXI_ACLK" TIG;
```

```
# Timing Ignore constraint on all signals that cross from S_AXI_ACLK domain to CAN_CLK domain
```

```
TIMESPEC "TS_SYS_CAN_TIG" = FROM "S_AXI_ACLK" TO "CAN_CLK" TIG;
```

The user must ensure that these default constraints are removed when both the clocks are driven from the same net.

I/O Constraints

I/O Standards

The pins that interface to the CAN PHY device have a 3.3 volt signal level interface. The following constraints can be used, provided the device I/O Banking rules are followed.

Select the I/O standards for the interface to the CAN PHY

```
INST "CAN_PHY_TX"          IOSTANDARD = "LVTTTL"
INST "CAN_PHY_RX"          IOSTANDARD = "LVTTTL"
```

Device Utilization and Performance Benchmarks

Core Performance

Table 40 shows example performance and resource utilization benchmarks for the Spartan®-6 FPGA (xc6slx45t-2-fgg484 device).

Table 40: Performance and Resource Utilization Benchmarks for the Spartan-6 FPGA (xc6slx45t-2-fgg484)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
2	2	0	374	553	794	74.582
2	2	1	405	648	866	68.474
2	2	2	311	651	896	79.930
2	2	3	394	654	878	78.616
2	2	4	386	648	864	72.427
4	4	0	378	561	805	61.755
4	4	1	398	664	893	76.383
4	4	2	385	667	897	73.089
4	4	3	389	670	891	74.974
4	4	4	393	664	886	74.438
8	8	0	368	585	826	71.058
8	8	1	403	680	895	74.789
8	8	2	384	683	909	71.235
8	8	3	392	686	912	82.597
8	8	4	365	680	902	77.042
16	16	0	391	601	848	69.959
16	16	1	402	696	919	83.605
16	16	2	411	699	925	75.120
16	16	3	401	678	905	73.228
16	16	4	430	696	905	69.832

Table 40: Performance and Resource Utilization Benchmarks for the Spartan-6 FPGA (xc6slx45t-2-fgg484)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
32	32	0	371	617	845	66.291
32	32	1	419	712	926	75.245
32	32	2	430	715	924	74.666
32	32	3	401	718	935	80.959
32	32	4	372	712	911	83.752
64	64	0	364	633	889	75.740
64	64	1	428	728	962	74.482
64	64	2	404	731	970	60.190
64	64	3	428	734	955	73.185
64	64	4	399	728	948	75.375

Table 41 shows example performance and resource utilization benchmarks for the Virtex®-6 FPGA (xc6vlx130t-2-ff484 device).

Table 41: Performance and Resource Utilization Benchmarks for Virtex-6 FPGA (xc6vlx130t-2-ff484)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
2	2	0	263	555	799	200.321
2	2	1	287	642	874	181.422
2	2	2	314	651	879	206.313
2	2	3	305	654	875	194.590
2	2	4	311	652	864	172.176
4	4	0	284	563	805	174.490
4	4	1	284	658	896	183.688
4	4	2	312	667	895	187.336
4	4	3	280	670	893	193.911
4	4	4	310	668	877	168.549
8	8	0	290	587	822	187.301
8	8	1	269	674	901	196.117
8	8	2	328	683	916	184.298
8	8	3	299	686	905	197.161
8	8	4	271	684	897	170.999
16	16	0	284	602	843	200.965
16	16	1	294	690	922	183.655
16	16	2	327	699	925	179.404
16	16	3	295	678	899	196.541
16	16	4	280	700	918	176.149
32	32	0	271	618	860	193.911
32	32	1	257	706	906	165.317
32	32	2	296	715	920	190.006
32	32	3	311	718	924	194.515
32	32	4	325	717	914	167.420
64	64	0	269	634	884	200.682
64	64	1	305	722	949	174.155
64	64	2	293	731	942	191.571
64	64	3	309	734	944	192.234
64	64	4	299	732	934	182.882

Table 42: Performance and Resource Utilization Benchmarks for Virtex-7 FPGA (xc7v285t-1-ffg1157)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
2	2	0	326	557	757	141.784
2	2	1	304	650	844	144.739
2	2	2	280	653	852	136.761
2	2	3	309	656	840	140.766
2	2	4	317	659	847	141.703
4	4	0	301	565	773	140.944
4	4	1	291	666	856	133.672
4	4	2	342	669	864	127.502
4	4	3	353	672	866	139.024
4	4	4	313	675	869	133.833
8	8	0	319	589	796	135.759
8	8	1	337	682	870	129.249
8	8	2	338	685	869	128.667
8	8	3	340	688	870	130.073
8	8	4	341	691	867	136.761
16	16	0	316	605	813	131.631
16	16	1	334	698	884	134.300
16	16	2	383	701	886	133.869
16	16	3	347	680	872	131.372
16	16	4	350	707	901	124.131
32	32	0	333	621	819	136.911
32	32	1	364	714	888	136.166
32	32	2	324	717	895	149.098
32	32	3	309	720	911	148.743
32	32	4	355	723	906	137.836
64	64	0	342	637	831	132.433
64	64	1	363	730	934	131.857
64	64	2	347	733	927	136.073
64	64	3	348	736	927	133.440
64	64	4	314	739	936	134.246

Table 43: Performance and Resource Utilization Benchmarks for Kintex-7 FPGA (xc7k410t-1-ffg900)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
2	2	0	339	557	758	148.214
2	2	1	372	650	844	145.264
2	2	2	362	653	851	144.279
2	2	3	356	656	849	134.048
2	2	4	369	659	854	146.156
4	4	0	341	565	767	151.561
4	4	1	338	666	861	122.459
4	4	2	366	669	858	144.051
4	4	3	359	672	867	141.323
4	4	4	355	675	862	124.517
8	8	0	330	589	788	105.742
8	8	1	345	682	861	140.213
8	8	2	355	685	874	149.298
8	8	3	352	688	870	142.025
8	8	4	342	691	879	153.822
16	16	0	339	605	803	133.511
16	16	1	350	698	886	141.523
16	16	2	349	701	899	141.824
16	16	3	341	680	868	138.523
16	16	4	370	707	894	142.878
32	32	0	337	621	822	148.148
32	32	1	389	714	890	146.284
32	32	2	375	717	905	133.458
32	32	3	378	720	898	136.073
32	32	4	373	723	903	137.099
64	64	0	345	637	825	139.723
64	64	1	388	730	920	148.456
64	64	2	373	733	929	139.567
64	64	3	384	736	925	136.874
64	64	4	365	739	927	124.922

Table 44: Performance and Resource Utilization Benchmarks for Artix-7 FPGA (xc7a355tdie-3)

Parameter Values			Device Resources			F _{MAX} (MHz)
C_CAN_RX_DPTH	C_CAN_TX_DPTH	C_CAN_NUM_ACF	Slices	Slice Flip-Flops	LUTs	AXI F _{MAX}
2	2	0	314	552	754	151.423
2	2	1	353	644	842	157.257
2	2	2	344	648	846	141.343
2	2	3	350	651	861	159.923
2	2	4	326	653	850	114.521
4	4	0	277	560	744	131.216
4	4	1	355	660	844	135.685
4	4	2	337	664	866	135.062
4	4	3	333	667	852	123.016
4	4	4	329	669	859	144.970
8	8	0	305	584	760	131.475
8	8	1	339	677	868	152.369
8	8	2	338	680	868	129.786
8	8	3	349	683	879	138.504
8	8	4	337	685	878	132.714
16	16	0	308	600	800	101.698
16	16	1	343	693	874	130.361
16	16	2	335	696	879	140.135
16	16	3	330	675	876	134.354
16	16	4	347	701	895	139.451
32	32	0	299	616	799	114.038
32	32	1	348	709	893	115.221
32	32	2	364	712	901	117.302
32	32	3	323	715	902	132.679
32	32	4	364	717	889	122.820
64	64	0	341	632	829	133.404
64	64	1	342	727	931	133.958
64	64	2	355	730	922	141.283
64	64	3	359	733	914	131.596
64	64	4	359	735	923	142.816

System Performance

To measure the system performance (F_{MAX}) of this core, this core was added as the Device Under Test (DUT) to Spartan-6 and Virtex-6 FPGA system as shown in Figure 4.

Because the AXI CAN core is used with other design modules in the FPGA, the utilization and timing numbers reported in this section are estimates only. When this core is combined with other designs in the system, the utilization of FPGA resources and timing of the design varies from the results reported here.

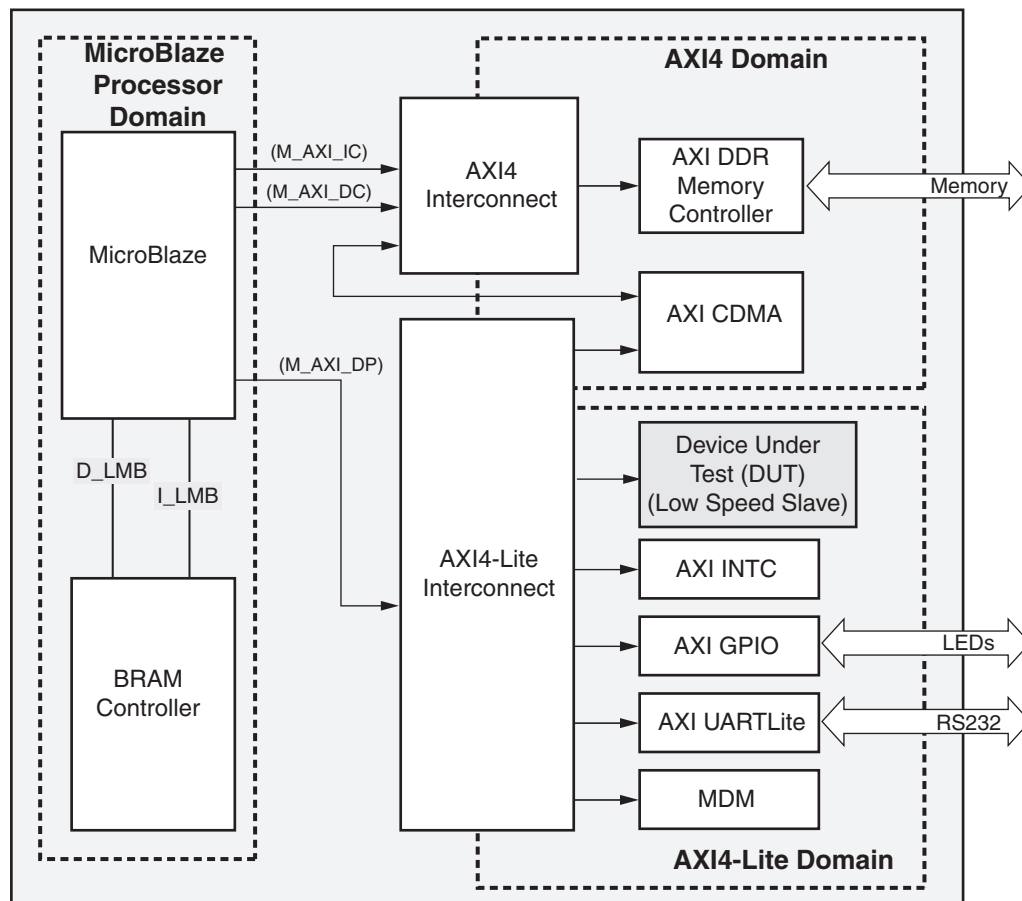


Figure 4: Spartan-6/Virtex-6 FPGA System with the AXI CAN Core as the DUT

The target FPGA was then filled with logic to drive the LUT and block RAM utilization to approximately 70% and the I/O utilization to approximately 80%. Using the default tool options and the slowest speed grade for the target FPGA, the resulting target F_{MAX} numbers are shown in Table 45.

Table 45: System Performance

Target FPGA	Target F_{MAX} (MHz)
S6LXT45-2	90
V6LXT130-1	180

The target F_{MAX} is influenced by the exact system and is provided for guidance. It is not a guaranteed value across all systems.

Support

Xilinx provides technical support for this LogiCORE IP product when used as described in the product documentation. Xilinx cannot guarantee timing, functionality, or support of product if implemented in devices that are not defined in the documentation, if customized beyond that allowed in the product documentation, or if changes are made to any section of the design labeled *DO NOT MODIFY*.

Reference Documents

1. *ISO 11898-1: Road Vehicles - Interchange of Digital Information - Controller Area Network (CAN) for High-Speed Communication.*
2. *Controller Area Network (CAN) version 2.0A and B Specification*, Robert Bosch GmbH
3. *AXI Interconnect IP Data Sheet (DS768)*

See www.xilinx.com/support/documentation/index.htm to find more Xilinx documentation.

Ordering Information

This Xilinx LogiCORE IP module is provided under the terms of the [Xilinx Core Site License](#). The core is generated using the Xilinx ISE® Embedded Edition software (EDK). For full access to all core functionality in simulation and in hardware, you must purchase a license for the core. Contact your [local Xilinx sales representative](#) for information on pricing and availability of Xilinx LogiCORE IP.

For more information, visit the [AXI CAN](#) product web page.

Information about this and other Xilinx LogiCORE IP modules is available at the [Xilinx Intellectual Property](#) page. For information on pricing and availability of other Xilinx LogiCORE IP modules and software, contact your [local Xilinx sales representative](#).

List of Acronyms

Acronym	Spelled Out
AFIR	Acceptance Filter ID Register
AFMR	Acceptance Filter Mask Register
AFR	Acceptance Filter Register
BRPR	Baud Rate Prescaler
BSP	Bit Stream Processor
BTL	Bit Timing Logic
BTR	Bit Timing Register
CAN	Controller Area Network
CRC	Cyclic Redundancy Check
DCM	Digital Clock Manager
DLC	Data Length Code
DUT	Device Under Test
DW	Data Word
ECR	Error Counter Register

Acronym	Spelled Out
ESR	Error Status Register
FIFO	First In First Out
FPGA	Field Programmable Gate Array
I	Industrial
I/O	Input/Output
ICR	Interrupt Clear Register
ID	Identifier
IDR	Identifier of the Received Message
IER	Interrupt Enable Register
IP	Intellectual Property
ISE	Integrated Software Environment
ISO	International Organization for Standardization
ISR	Interrupt Status Register
LLC	Logical Link Control
LSB	Least Significant Bit
MAC	Media Access Controller
Mb/s	Megabits per second
MHz	Mega Hertz
MSB	Most Significant Bit
MSR	Mode Select Register
RTR	Remote Transmission Request
RX	Reception
SJW	Synchronization Jump Width
SOF	Start Of Frame
SR	Status Register
SRR	Software Reset Register
TX	Transmission
UAF	Use Acceptance Filter
VHDL	VHSIC Hardware Description Language (VHSIC an acronym for Very High-Speed Integrated Circuits)
XPS	Xilinx Platform Studio (part of the EDK software)
XST	Xilinx Synthesis Technology

Revision History

Date	Version	Revision
09/21/10	1.0	First release of the core with AXI interface support. The previous release of this document was ds265.
09/21/10	1.0.1	Documentation only. Added inferred parameters text on page 7.
06/22/11	2.0	Updated for core version 1.03.a and 13.2 tools. Added new supported architectures.

Notice of Disclaimer

The information disclosed to you hereunder (the “Materials”) is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available “AS IS” and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of the Limited Warranties which can be viewed at <http://www.xilinx.com/warranty.htm>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in Critical Applications: <http://www.xilinx.com/warranty.htm#critapps>.